CONVERGE CHALLENGE · BRIEFING DO TIME · DOCUMENTO 2 DE 3

Dossiê Técnico

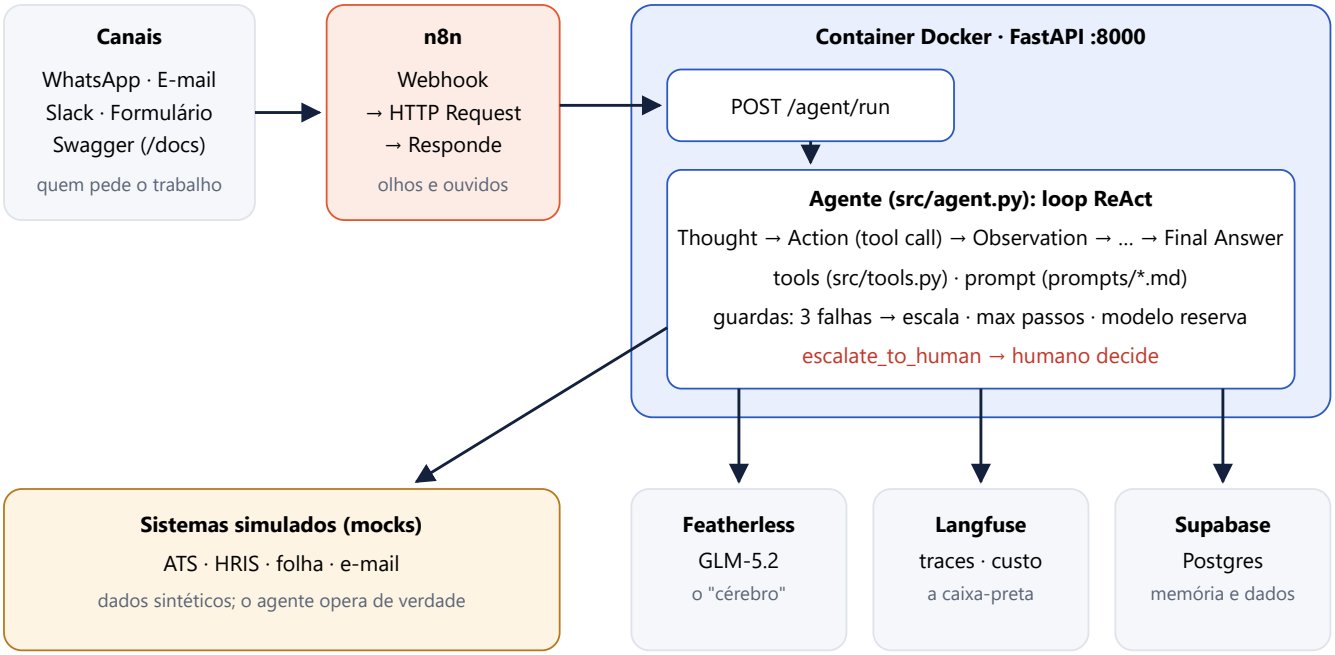
Cada peça da nossa arquitetura: o que é, o que faz no projeto, como explicar para a banca em uma frase, e as armadilhas que já pegaram a gente.

02

Neste documento

1. A arquitetura em uma página
2. As peças: Docker, FastAPI, Agente, Featherless, Langfuse, Supabase, n8n, túnel/deploy, MCP, Git
3. Quanto custa (números medidos)
4. Cola de comandos
5. Glossário

1 A arquitetura em uma página



Fluxo de uma tarefa: alguém pede algo num canal → o n8n recebe e repassa para a API → o agente pensa, chama ferramentas sobre os sistemas (simulados) e o LLM decide o próximo passo → a resposta volta pelo n8n. Cada passo vira um *trace* no Langfuse e cada execução vira uma linha no Supabase (`agent_runs`).

> **Já validado de ponta a ponta (06/11)**
>
> `POST localhost:5678/webhook/agente` → n8n → `api:8000/agent/run` → GLM-5.2 chama a ferramenta `get_current_time` → resposta correta em 10,6 s por US$ 0,0022.

2 As peças

INFRA

Docker + Docker Compose: a caixa de isolamento

Para a banca: "O sistema roda igual em qualquer máquina e em produção, porque viaja empacotado com tudo de que precisa."

O que é	O `Dockerfile` é a receita (Python 3.12 + dependências + código). O `docker-compose.yml` liga vários containers juntos.
No projeto	Perfis: `api` (padrão), `n8n` (API + n8n na porta 5678), `local-db` (Postgres local de reserva). O código em `src/` é montado no container com `--reload`, então editar o arquivo atualiza a API sozinho.
Armadilhas vividas	**"failed to connect to the docker API"** → o Docker Desktop está fechado; abra o app. **Mudou o `.env`?** O container só lê ao subir: `docker compose up -d --force-recreate`. Dentro do compose, a API se chama `http://api:8000`. Do seu navegador, é `http://localhost:8000`.

API

FastAPI + Swagger: a porta de entrada

Para a banca: "Qualquer sistema da empresa pode delegar trabalho ao agente por uma API padrão."

Rotas	`GET /health` mostra o que está configurado (LLM, Supabase, Langfuse e lista de tools). `POST /agent/run` recebe `{"task", "context", "session_id", "tags"}` e devolve status, resposta, passos, tokens, custo e `trace_url`.
Swagger	`http://localhost:8000/docs`: botão *Try it out* → *Execute*. Gerado automaticamente.
Armadilha vivida	**405 Method Not Allowed** = você abriu `/agent/run` pela barra do navegador (que faz um GET). A rota é POST: use o Swagger, o n8n ou o curl.

NÚCLEO

O Agente: loop ReAct com tool calling

Para a banca: "Não é um script: o agente decide a cada passo qual ferramenta usar, observa o resultado, se corrige e sabe quando parar e chamar um humano."

Ciclo	**Thought** (raciocínio) → **Action** (chamada de ferramenta, com argumentos validados por Pydantic) → **Observation** (resultado ou erro devolvido ao modelo) → repete → **Final Answer**.
Tool calling	O modelo devolve um JSON estruturado dizendo qual função chamar e com quais argumentos. **Confirmamos que o GLM-5.2 faz isso nativamente.**
Guardas	Limite de passos (`AGENT_MAX_STEPS`) · 3 falhas seguidas de ferramenta → `status="escalated"` · tool `escalate_to_human` para decisões sobre pessoas e dados faltando · campos inventados pelo modelo são rejeitados (`ToolArgs`).
Status finais	`completed` · `escalated` · `max_steps_reached` · `error`
Onde mexer	Tools novas em `src/tools.py` , prompt em `prompts/system_prompt.md` . Siga a skill `building-react-agents` .

LLM

Featherless: o cérebro (gateway de LLM)

Para a banca: "Usamos modelos abertos via API compatível com OpenAI: dá para trocar de modelo em uma linha e controlamos o custo por execução."

Modelos	Principal: `zai-org/GLM-5.2` · Reserva automática (em erro 400/503): `Qwen/Qwen3-32B` · Barato para texto simples: `google/gemma-4-31B-it` .
Créditos	US$ 25 por pessoa, cada um com a própria chave. **Cancelem o plano antes da primeira cobrança** se não forem continuar usando.
Concorrência	O limite é de "unidades concorrentes": modelos grandes ocupam mais. Muitas chamadas em paralelo → **429**.
Armadilha vivida	O GLM é um modelo de raciocínio: com `max_tokens` baixo, a resposta pode vir **vazia**. Um "pong" gastou 128 tokens de saída.
Erros	400 modelo frio · 401 chave · 403 desbloquear modelo · 429 concorrência · 503 sem capacidade (retry, depois reserva).

AUDITORIA

Langfuse: a caixa-preta

Para a banca: "Cada decisão do agente fica registrada: dá para auditar o que ele fez, por quê, quanto custou e quanto demorou."

No projeto	Um *trace* por execução: `agent.run` → `llm.featherless` (modelo, tokens, US$) → `tool.execute`. A resposta da API traz `trace_url` para abrir direto.
Chaves	Langfuse → seu projeto → *Settings* → *API Keys*. **Uma chave real tem cerca de 40 caracteres.** O host precisa ser da região do projeto (EU `cloud.langfuse.com` / US `us.cloud.langfuse.com`).
Armadilha vivida	As chaves placeholder (`pk-lf-…`) derrubavam a API com 500. **Já corrigido:** sem chave válida, o tracing desliga sozinho e a API segue funcionando. Interruptor: `LANGFUSE_ENABLED=false`. **Nunca comente o `@observe`** para "consertar" um erro.
Demo Day	Deixe uma aba aberta em Tracing filtrado por `tags = demo` e mostre a árvore de decisão ao vivo.

DADOS

Supabase: memória e dados de negócio

Para a banca: "Todo o histórico fica num Postgres gerenciado: sessões, execuções e os dados que o agente opera."

O que é	Postgres na nuvem, com API, autenticação e painel. É uma alternativa aberta ao Firebase.
No projeto	Tabela `agent_runs` (trilha de auditoria: tarefa, status, passos, tokens, custo, trace). Depois virão as tabelas do domínio, como colaboradores e férias sintéticos.
Chave	`SUPABASE_KEY` = *service_role*: só no backend, ignora as regras de acesso (RLS). Nunca vai para o front-end nem para o n8n.
Pendente	A tabela ainda não existe: rode `sql/001_agent_runs.sql` no *SQL Editor*. Sem ela a API funciona, mas não salva o histórico.

INTEGRAÇÕES

n8n: os olhos e ouvidos

Para a banca: "O agente entra nos canais que a empresa já usa (WhatsApp, e-mail, Slack) sem a gente reescrever integrações."

O que é	Automação visual com centenas de conectores prontos, como o Zapier, mas com opção de rodar localmente.
No projeto	O fluxo pronto `n8n/agent-bridge.json` faz Webhook → `POST api:8000/agent/run` → responde com status, resposta, custo e trace. **Já testado.**
Divisão de papéis	O n8n também tem um nó "AI Agent", mas mantenha o **cérebro em Python**: lá ele tem testes, trace no Langfuse e guardas. O n8n só cuida dos canais.
Armadilha vivida	No n8n 2.x, o fluxo precisa ser **publicado** (`publish:workflow`), não só importado. Sem isso: "webhook not registered".

ACESSO EXTERNO

Túnel (ngrok / cloudflared) × deploy

Para que serve: expor o `localhost:8000` com uma URL pública, para um n8n na nuvem ou um WhatsApp conseguir chamar o notebook.

ngrok	Precisa de conta e *authtoken*. `ngrok http 8000` . No plano gratuito a URL pode mudar a cada vez que você reinicia.
cloudflared	`cloudflared tunnel --url http://localhost:8000` gera uma URL temporária sem login. Bom para testes rápidos.
Importante	A submissão pede um **link navegável e testável pelos jurados**. Túnel para o notebook é frágil (Wi-Fi cai, notebook hiberna). Para a entrega, o ideal é **deploy** do container num serviço que rode Docker (Render, Railway, Fly.io, Google Cloud Run...). Testem o deploy cedo.

OPCIONAL

MCP (Model Context Protocol): o "USB" das ferramentas de IA

O que é de verdade: um protocolo aberto (criado pela Anthropic, hoje adotado por vários fornecedores) para um *servidor* expor ferramentas e dados a qualquer *cliente* de IA (Claude, ChatGPT, IDEs) num formato padrão.

Correção	O nosso agente **não usa MCP hoje**. As ferramentas são registradas no `ToolRegistry` e enviadas ao modelo pelo *tool calling* da API OpenAI. É o mesmo conceito (nome + descrição + schema JSON), sem o protocolo.
Onde poderia entrar	(1) Expor as nossas tools como servidor MCP, para que um RH use o "digital worker" de dentro do Claude/ChatGPT. (2) Consumir servidores MCP prontos (Gmail, Slack, Postgres) em vez de escrever integrações.
Recomendação	Não é necessário para vencer. Só vale como extra se o núcleo já estiver sólido, e aí pode virar um argumento de "plugável em qualquer IA" no pitch.

TIME

Git/GitHub + skills: trabalhar em 2 a 4 pessoas

Fluxo	Commits pequenos e frequentes direto na `main` (ou branches curtas). Dê `git pull` antes de começar cada bloco. **O histórico de commits pode ser pedido como evidência** de que o código foi feito durante o evento.
Divisão	Cada pessoa com arquivos diferentes (ex.: uma em `tools.py`, outra nos mocks e dados, outra no n8n e deploy) evita conflitos.
Skills	Claude Code e Codex leem `AGENTS.md` e as 3 skills sozinhos. No ChatGPT web, suba os arquivos `skills/*.md` num Projeto (ver `skills/README.md`).

3 Quanto custa (números medidos)

Medição real (06/11/2026)	Tokens	Custo
Chamada simples ao GLM-5.2 ("pong")	18 entrada + 128 saída	US$ 0,00059
Execução completa via n8n, com 1 ferramenta (2 chamadas ao LLM)	n/d	**US$ 0,0022**

Projeção (estimativa)

Uma execução realista do produto, com 4 a 8 ferramentas e mais contexto, deve ficar em torno de **US$ 0,01 a 0,05** (5 a 25× a medida acima). Com os **US$ 100 do time** (4 × 25), isso dá **2.000 a 10.000 execuções**, bem mais do que o hackathon vai usar.

Para o pitch: "Cada caso processado custa cerca de R$ 0,10–0,30 em IA, contra X minutos de um analista de DP a R$ Y por hora." Medir o custo real do caso da demo no domingo de manhã (o Langfuse e o campo `cost_usd` mostram).

4 Cola de comandos

```
# Ambiente (uma vez por máquina)
git pull
python -m venv .venv && .venv\Scripts\activate       # Windows
pip install -r requirements.txt                        # "No module named langfuse" = venv
não ativado
copy .env.example .env                                 # preencher as chaves

# Conferir tudo
python -m scripts.check_setup                          # Featherless, tool calling,
Langfuse, Supabase

# Rodar
docker compose up --build                              # só a API → localhost:8000/docs
docker compose --profile n8n up -d --build             # API + n8n → localhost:5678
docker compose up -d --force-recreate                  # depois de mudar o .env
docker compose logs -f api                             # ver o que o agente está fazendo

# n8n: ponte (1x)
docker compose exec n8n n8n import:workflow --input=/workflows/agent-bridge.json
docker compose exec n8n n8n publish:workflow --id=AgentBridge00001
docker compose restart n8n

# Testar a ponte
curl -X POST http://localhost:5678/webhook/agente -H "Content-Type: application/json" -d "
{\"task\":\"Que horas são em Recife?\"}"
```

5 Glossário

Agente / digital worker	Sistema que recebe um *objetivo* e conduz o processo sozinho, usando ferramentas, até o resultado ou até precisar de um humano.
ReAct	Padrão Reasoning + Acting: alternar raciocínio e ação, observando cada resultado (Yao et al., 2022).
Tool calling	O modelo devolve "chame a função X com os argumentos Y" em JSON estruturado, em vez de texto.
Human-in-the-loop	Pontos em que o agente para e um humano decide. Obrigatório para decisões sobre pessoas.
Token	Pedaço de texto (cerca de ¾ de uma palavra). É a unidade de cobrança dos LLMs, separada em entrada e saída.
Trace / span	Trace é o registro de uma execução inteira. Span é cada passo dentro dela (chamada ao LLM, ferramenta).
Webhook	URL que recebe um aviso (POST) quando algo acontece em outro sistema.
Mock	Versão simulada de um sistema externo (ATS, HRIS, folha), com dados sintéticos.

Idempotência	Repetir a mesma ação não duplica o efeito. É essencial para agentes que tentam de novo após um erro.
Eval	Conjunto fixo de casos de teste para medir se o agente acerta. Separa "funciona" de "teve sorte".
RLS / service_role	Regras de acesso por linha do Supabase / chave de backend que ignora essas regras.
eSocial	Sistema do governo onde as empresas informam eventos trabalhistas (admissão, férias, desligamento).